AIORI-3 HACKATHON TECHNICAL PROPOSAL

PS-011: Authorised Attack-Surface Inventory and Change Monitor

Category: Cybersecurity | **Track:** Internet Operations & Attack Surface Management

1. Executive Summary & Problem Overview

Modern cloud environments rapidly expand, leading to accumulated exposed digital assets: unmaintained test subdomains, expired TLS certificates, rogue open service ports, and dangling DNS CNAME records pointing to decommissioned multi-cloud instances. Such hygiene oversights expose organisations to security degradation, subdomain takeovers, and unintended service exposures. This project implements an **Authorised Attack-Surface Inventory and Change Monitor** designed for up to 20 lab domains and 10 hosts, providing automated continuous discovery, baseline drift detection, strict scope enforcement, and actionable alerts without aggressive exploitation.

2. System Architecture & Component Design

Component	Key Responsibilities	Security & Operational Controls
Scope Guard (core/scope_guard.py)	Enforces strict boundary checking on domains and IPs. Hard limit of 20 domains / 10 hosts.	Rejects out-of-scope targets with ScopeViolationError. Enforces non-exploitation safety policy.
4 Documented Checks (checks/service_inspector.py)	Modular multi-layer port & service inspection across TCP, Banners, TLS, and Application HTTP.	Non-flooding, standard RFC handshakes, passive banner listening, and SNI TLS negotiation.
DNS & Dangling Engine (checks/dns_inspector.py)	Catalogs A, AAAA, CNAME, MX, TXT, NS, SOA records. Analyzes CNAMEs against 9 cloud providers.	Tests NXDOMAIN and unlinked cloud signatures (AWS S3, GitHub, Azure, Heroku) without hostile takeovers.
Diff & Baseline Engine (core/diff_engine.py)	Computes delta between current scan and golden baseline snapshot. Flags asset, port, cert drift.	Categorizes severity (Critical/High/Medium/Low/Info). Triggers alerts for unexpected services.
False-Positive Analyzer (analysis/false_positive.py)	Scores confidence (High/Med/Low) and FP risk against CDNs, wildcard DNS, and SYN-proxies.	Provides human-readable technical triage rationales and precise verification steps.

3. The 4 Documented Port & Service Checks

Check Designation	Protocol Layer & Target	Inspection Methodology & Outputs
Check 1: TCP Port State & Latency	Transport (L4) / RFC 793	Performs standard TCP 3-way handshake with configurable timeout. Outputs state (OPEN/CLOSED/FILTERED) and RTT latency in milliseconds.
Check 2: Service Banner Grab	Session / Presentation (L5-L6)	Passively reads service greeting (up to 1024 bytes); sends gentle probe if quiet. Regex fingerprinting for SSH, SMTP, FTP, Redis, MySQL.
Check 3: TLS/SSL Handshake	Cryptographic Transport (L6)	Initiates SNI ClientHello. Negotiates TLSv1.2/1.3 and cipher suite. Parses DER X.509 cert: computes days-to-expiry, issuer, SANs, and expired state.

Check 4: HTTP/HTTPS Inspection	Application (L7) / RFC 9110	Transmits authorized GET probe. Analyzes HTTP status, Server/X-Powered-By headers, HTML <title>, and defense headers (HSTS, CSP, X-Frame-Options).

4. Seeded-Issue Detection Results & Verification

To satisfy the hackathon evaluation mandate, the system includes a built-in Lab Harness with verified detections:

Seeded Vulnerability	Test Asset	Detection Mechanism & Severity	FP Risk & Confidence
Expired TLS Certificate	expired-tls.aiori.internal	**CRITICAL:** Cryptographic comparison of not_after_utc against current UTC clock. Detected expired 30 days ago.	Confidence: **HIGH** FP Risk: **LOW**
Unexpected Open Service	10.0.1.15:8888	**HIGH:** Baseline policy comparison flags unapproved port. Checks 2 & 4 confirm active Werkzeug debug HTTP server.	Confidence: **HIGH** FP Risk: **LOW**
Dangling DNS (AWS S3)	orphaned-bucket.aiori.internal	**HIGH:** CNAME target points to S3. Non-intrusive probe detects XML signature: NoSuchBucket with HTTP 404.	Confidence: **HIGH** FP Risk: **LOW**
Dangling DNS (GitHub)	blog.aiori.internal	**HIGH:** CNAME target points to *.github.io. Probe confirms There isn't a GitHub Pages site here.	Confidence: **HIGH** FP Risk: **LOW**
False-Positive Control	test-cdn.aiori.internal	**CONTROL:** CNAME points to CloudFront edge returning 404. Filter recognizes multi-tenant CDN routing behavior.	Confidence: **LOW** FP Risk: **HIGH**

5. False-Positive Analysis & Confidence Framework

A central requirement of AIORI-3 PS-011 is reporting false positives and acknowledging that not all dangling records or open ports are exploitable. Our engine applies multi-factor triage filters:

- **Multi-Tenant CDN vs Orphaned Resource:** Distinguishes between dedicated storage buckets (S3) where 404 NoSuchBucket indicates true orphaned namespaces, versus shared CDN edges (CloudFront/Cloudflare) where 404 is standard when host routing is unconfigured.
- **Wildcard DNS Filtering:** Compares resolutions against random non-existent test labels (e.g. random_test_xyz.domain) to prevent false alerts on catch-all zones.
- **Firewall SYN-Proxy Sinkholes:** Evaluates total open port density. If >15 ports open indiscriminately, marks alerts as Low Confidence / High FP Risk (honeypot or port reflector).
- **SNI Isolation:** Enforces SNI in all TLS handshakes to avoid false expired cert alerts from fallback default certificates on reverse proxies.

6. Prototype Demonstration with Pseudocode Snippet (Mentor Demonstration)

The following structured pseudocode represents the operational core of the change detection and inspection engine:

```
FUNCTION RunAttackSurfaceMonitorCycle(scopeConfig, isBaseline):
    # 1. Enforce Organiser-Approved Boundaries
    domains = ScopeGuard.FilterDomains(scopeConfig.domains, maxLimit=20)
    hosts = ScopeGuard.FilterHosts(scopeConfig.hosts, maxLimit=10)

    # 2. Multi-Layer Discovery
    snapshot = NewInventorySnapshot(timestamp=NOW_UTC())
    FOR EACH domain IN domains:
        dnsRecords = DNSInspector.CatalogRecords(domain, [A, AAAA, CNAME, MX, TXT])
        FOR EACH record IN dnsRecords WHERE record.Type == CNAME:
            isDangling, provider, reason = DNSInspector.CheckDanglingCNAME(record.Target)
            IF isDangling: record.FlagAsDanglingCandidate(provider, reason)
        tlsCert = TLSInspector.Probe(domain, port=443)
        snapshot.AddDomain(domain, dnsRecords, tlsCert)

    FOR EACH host IN hosts:
        FOR EACH port IN scopeConfig.approvedPorts:
            # The 4 Documented Checks
```

```
        pState, rtt = Check1_TCPConnect(host, port)
        IF pState == OPEN:
            banner, service = Check2_BannerGrab(host, port)
            hasTLS, tlsVer, cipher, cert = Check3_TLSNegotiation(host, port)
            httpData = Check4_HTTPInspection(host, port, hasTLS)
            isUnexpected = (port NOT IN scopeConfig.baselineServices[host])
            snapshot.AddService(host, port, service, isUnexpected, cert, httpData)

# 3. Baseline Delta & False-Positive Triage
baseline = Storage.LoadBaseline()
diff = DiffEngine.Compare(baseline, snapshot)
FOR EACH alert IN diff.Alerts:
    alert.Confidence, alert.FPRisk, alert.Triage = FalsePositiveAnalyzer.Evaluate(alert)
    AlertManager.Dispatch(alert)

Storage.SaveSnapshot(snapshot, markBaseline=isBaseline)
RETURN diff
```

7. Ethical Scope Guardrails & References

Strict Scope Enforcement: All probes are constrained to written, organiser-approved targets. **No Exploitation:** Probes are purely diagnostic and passive; no takeover claims or payload deliveries are made.

References: [1] OWASP Web Security Testing Guide (WSTG-INFO-04 to 08); [2] RFC 793 (TCP); [3] RFC 8446 (TLS 1.3); [4] RFC 1035 (DNS).